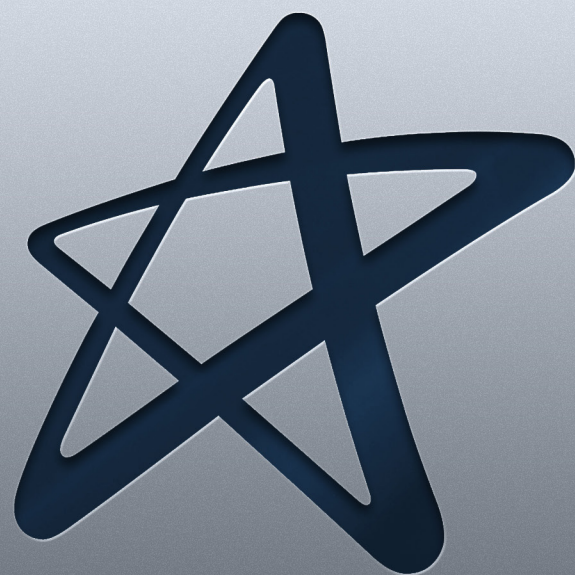


Estrutura de Dados Lineares



Cruzeiro do Sul Virtual
Educação a distância

Material Teórico



Algoritmos de Classificação em Tempo Linear

Responsável pelo Conteúdo:

Prof. Dr. Luciano Rossi

Revisão Textual:

Mateus Gonçalves Santos

UNIDADE

Algoritmos de Classificação em Tempo Linear



- Introdução;
- Ordenação por Contagem (*Counting Sort*);
- Ordenação por Balde (*Bucket Sort*);
- Fundamentos de Análise de Complexidade de Algoritmos.



OBJETIVOS DE APRENDIZADO

- Apresentar ao aluno os principais algoritmos de ordenação em tempo linear;
- Compreender, de maneira intuitiva, o conceito de complexidade de algoritmos a partir da comparação entre as diferentes estratégias de ordenação.

Orientações de estudo

Para que o conteúdo desta Disciplina seja bem aproveitado e haja maior aplicabilidade na sua formação acadêmica e atuação profissional, siga algumas recomendações básicas:



Assim:

- ✓ Organize seus estudos de maneira que passem a fazer parte da sua rotina. Por exemplo, você poderá determinar um dia e horário fixos como seu “momento do estudo”;
- ✓ Procure se alimentar e se hidratar quando for estudar; lembre-se de que uma alimentação saudável pode proporcionar melhor aproveitamento do estudo;
- ✓ No material de cada Unidade, há leituras indicadas e, entre elas, artigos científicos, livros, vídeos e sites para aprofundar os conhecimentos adquiridos ao longo da Unidade. Além disso, você também encontrará sugestões de conteúdo extra no item **Material Complementar**, que ampliarão sua interpretação e auxiliarão no pleno entendimento dos temas abordados;
- ✓ Após o contato com o conteúdo proposto, participe dos debates mediados em fóruns de discussão, pois irão auxiliar a verificar o quanto você absorveu de conhecimento, além de propiciar o contato com seus colegas e tutores, o que se apresenta como rico espaço de troca de ideias e de aprendizagem.

Introdução

Nas unidades anteriores foram analisados diferentes algoritmos de ordenação. O *Bubble Sort*, por exemplo, percorre todo o arranjo comparando pares contíguos de elementos de modo a isolar o maior ou menor elemento, dependendo da versão do algoritmo, em uma das extremidades do arranjo. Na sequência, o procedimento se repete desconsiderando o subarranjo já ordenado. De maneira similar, o algoritmo *Insertion Sort* seleciona um elemento no arranjo, de modo a dividi-lo em dois subarranjos: um ordenado à esquerda e outro não ordenado à direita. O elemento selecionado é comparado com todos os elementos no subarranjo à esquerda, buscando por sua posição ordenada nesse segmento.

Uma característica comum aos algoritmos *Bubble Sort* e *Insertion Sort* é que eles comparam várias vezes um elemento com os demais elementos no arranjo, resultando em muitas comparações para a ordenação do arranjo. Nesse sentido, uma comparação entre dois elementos demora um determinado tempo para ser realizada, assim, quanto mais comparações são feitas, mais o algoritmo demora em terminar sua execução.

Os algoritmos de classificação em tempo linear são aqueles que não realizam comparações entre todos os elementos no arranjo. Assim, o número de instruções que são realizadas é proporcional à quantidade de elementos pertinentes ao arranjo. Veja que, se considerarmos um arranjo com n elementos, os algoritmos *Bubble Sort* e *Insertion Sort* executam um número de instruções proporcional a $n \times n$, ou n^2 , pois cada elemento será comparado com os demais elementos no arranjo.

Nesta unidade avaliaremos os algoritmos que realizam a ordenação de um arranjo executando um número de instruções proporcional a n . Esses algoritmos requerem algumas restrições prévias, como veremos a seguir.

Ordenação por Contagem (*Counting Sort*)

O algoritmo de ordenação por contagem, ou *Counting Sort*, executa um número de instruções proporcional a n . A premissa para esse algoritmo é que a entrada seja um arranjo de inteiros maiores ou iguais a 0 e menores ou iguais a k . Nesse sentido, é preciso que se conheça previamente o maior valor no arranjo.



Veja uma animação sobre o funcionamento do algoritmo *Counting Sorting*.
Disponível em: <https://youtu.be/7zuGmKfUt7s>

O algoritmo 1 descreve as instruções do *Counting Sort*, o qual recebe como parâmetros de entrada um arranjo de inteiros positivos (A), um arranjo vazio (B) que receberá os elementos de A em ordem crescente e o maior valor k em A .

Inicialmente, na linha 1 do algoritmo 1, é criado um novo arranjo (C) com posições no intervalo entre 0 e k . Note que os índices do arranjo C correspondem aos valores possíveis de serem observados no arranjo A , visto que há a premissa de que os valores

em A devem ser inteiros positivos e menores ou iguais a k . Essa é a ideia chave para o funcionamento do algoritmo *Counting Sort*, cada posição no arranjo C representa um valor no arranjo A . Dessa forma o arranjo C pode ser utilizado para armazenar as quantidades de cada valor observado no arranjo A .

O laço na linha 2 do algoritmo é responsável por percorrer o arranjo C e atribuir a cada posição o valor zero. Em seguida, o laço na linha 4 percorre o arranjo A , de modo que os valores em A sejam utilizados como indexadores das posições em C , possibilitando a contagem daqueles valores.

Considere a Figura 1a, o primeiro valor em A é $A[1] = 2$, assim na posição 2 do arranjo C o valor é incrementado em uma unidade ($C[2] = 1$) indicando que o valor 2 foi observado uma vez no arranjo A . O próximo valor é $A[2] = 5$ que será representado no arranjo C por meio do incremento do valor desta posição ($C[5] = 1$). Após a finalização do laço nas linhas 4 e 5, o arranjo C estará preenchido da seguinte forma: $C[0] = 2$, $C[1] = 0$, $C[2] = 2$, $C[3] = 3$, $C[4] = 0$ e $C[5] = 1$. Veja que $C[0] = 2$ indica que o valor 0 aparece duas vezes no arranjo A , do mesmo modo, $C[1] = 0$ sinaliza que o valor 1 não está presente no arranjo A .

O laço, que é descrito nas linhas 6 e 7 do algoritmo 1, é responsável por acumular as quantidades de observações no arranjo C . Por exemplo, a posição $C[1]$ acumula o seu próprio valor e o valor de seu antecessor $C[0]$, ou seja, $C[1] = C[1] + C[0]$. O mesmo ocorre com a posição $C[2] = C[2] + C[1]$ e assim sucessivamente até $C[k] = C[k] + C[k-1]$.

A acumulação dos valores no arranjo C tem por objetivo identificar qual é a posição de inserção de um valor em A , que é representado pelos indexadores em C , no arranjo B . Considere o arranjo C acumulado, que está representado na Figura 1a. Veja que para o índice 0 o valor acumulado é igual a 2, indicando que o valor 0 ocupará as posições 2 e 1 no arranjo B . O índice 1, no arranjo C , também tem um valor acumulado igual a 2, isso ocorre pois não há o valor 1 no arranjo A , consequentemente nenhuma inserção será feita em B . O índice 2 tem valor acumulado igual a 4 no arranjo C , isso indica que esse valor ocupará as posições 4 e 3 no arranjo B . Essa lógica para a localização das posições no arranjo B se repete para as demais posições.

Algoritmo 1: Algoritmo *Counting Sort*

CountingSort (A, B, k)

1. seja $C[0 \dots k]$ um novo arranjo
2. **para** $i \leftarrow 0$ **até** k **faça**
3. $C[i] \leftarrow 0$
4. **para** $j \leftarrow 1$ **até** $|A|$ **faça**
5. $C[A[j]] \leftarrow C[A[j]] + 1$
6. **para** $i \leftarrow 1$ **até** k **faça**
7. $C[i] \leftarrow C[i] + C[i - 1]$
8. **para** $j \leftarrow |A|$ **até** 1 **faça**
9. $B[C[A[j]]] = A[j]$
10. $C[A[j]] \leftarrow C[A[j]] - 1$

O último laço, descrito nas linhas 8 – 10 no algoritmo 1, é responsável por inserir os valores, de forma ordenada, no arranjo de destino B . Note que o arranjo A é percorrido do fim para o início, sendo que a notação $|A|$ indica o número de elementos em A .

O valor da posição $A[8]$, que é a última posição do arranjo A , conforme ilustrado na Figura 1b, será utilizado como índice do arranjo C , nesse caso temos que $A[8] = 3$ então $C[3] = 7$ será utilizado como índice no arranjo B que receberá o valor de A , ou seja, $B[7] = A[8]$. Após a inserção do valor 3 na posição 7 no arranjo B , é necessário decrementar o valor acumulado em C , visto que aquela posição já foi ocupada e quando outro valor 3 for identificado ele deverá ocupar a posição 6 no arranjo B (linha 10).



Figura 1 – Passo a passo da execução do algoritmo *Counting Sort*

Fonte: Adaptado de CORMEN, 2009

A Figura 1 ilustra o passo a passo da execução do algoritmo *Counting Sort*, que recebe o arranjo $A = [2, 5, 3, 0, 2, 3, 0, 3]$ como entrada. No detalhe (a) temos, além do arranjo de entrada, o arranjo C em dois estados. No primeiro estado, os valores, em cada posição de C , se referem à frequência com a qual o valor, representado pelo índice da posição, é observado em A . O segundo estado de C representa as frequências acumuladas.

No detalhe (b), o percurso no arranjo A se inicia pela última posição, que é apontada pela variável j . Veja que, a partir do valor dessa posição, se localiza a posição no arranjo C , cujo valor é utilizado como posição de inserção no arranjo B . Nesse contexto, temos $j = 8$, $A[j] = 3$, $C[A[j]] = 7$ e $B[C[A[j]]] = A[j]$. Essa dinâmica resulta na inserção do valor 3 na posição $B[7]$, a qual corresponde a posição do valor em ordem no arranjo. Após a inserção do valor no arranjo B , a frequência acumulada na posição 3 de C é decrementada em uma unidade, de modo a referenciar a posição de inserção em C do próximo valor 3 presente em A .

O detalhe (c) na Figura 1 ilustra a inserção do próximo elemento. O elemento em questão está armazenado na posição $j = 7$ no arranjo A , ou seja, $A[j] = 0$. Da mesma maneira observada na inserção anterior, esse valor é utilizado para indicar a posição no arranjo C ($C[0] = 2$) cujo valor será utilizado como índice da posição em B ($B[2]$), a qual receberá o valor de A , ou seja, $B[2] = 0$. O valor 2 na posição 0 em C é decrementado para 1 que indica a posição na qual será armazenado o próximo valor 0 observado em A .

O processo descrito anteriormente se repete até que todos os valores sejam inseridos no arranjo B , de maneira ordenada, cujo estado final é representado no detalhe (i) da Figura 1.



Por que o *Counting Sort* é classificado como um algoritmo de classificação em tempo linear?

Considere que o tamanho do arranjo A e o maior valor observado nesse arranjo é k e ambos são, no máximo, iguais a n , ou seja, $|A|, k \leq n$. Analisando o algoritmo 1, é possível verificar a presença de quatro laços de repetição, nas linhas 2, 4, 6 e 8, note que o número de repetições executadas em cada laço é menor ou igual a n e suponha, ainda, que o número máximo de instruções em cada laço é representado por c . Nesse contexto, podemos afirmar que o algoritmo executará, no máximo, $4 \times n \times c$ instruções.

O algoritmo de ordenação *Bubble Sort*, visto na unidade 2 desta disciplina, apresenta dois laços de repetição aninhados. Considerando as mesmas premissas anteriores, ou seja, $|A| \leq n$ e o número máximo de instruções no laço é representado por c , podemos afirmar que o número máximo de instruções que serão executadas é proporcional a $n \times n \times c$ ou $n^2 \times c$.

A título de comparação, vamos considerar que ambos os algoritmos realizam somente uma única instrução em cada laço, assim $c = 1$. A Figura 2 apresenta um diagrama no qual o eixo horizontal representa o tamanho do vetor a ser ordenado (n) e o eixo vertical representa o número de instruções executadas ($f(n)$). As curvas azul e vermelha representam o número de instruções executadas pelo *Counting Sort* ($4n$) e *Bubble Sort* (n^2), respectivamente. Veja que para um arranjo com quatro elementos, o número de instruções executadas é 16 para ambos os algoritmos. A partir desse tamanho de arranjo, o número de instruções executadas pelo *Bubble Sort* cresce mais rapidamente que o número de instruções executadas pelo *Counting Sort*. Por exemplo, se o arranjo tiver 1.000 elementos, o *Bubble Sort* executará 1 milhão de instruções, enquanto que o *Counting Sort* executará quatro mil instruções para ordenar o mesmo arranjo.



Figura 2 – Diagrama representando as curvas de funções linear e quadrática

Fonte: Acervo do conteudista

A descrição comparativa entre o número de instruções executadas pelos algoritmos *Bubble Sort* e *Counting Sort* tem por objetivo apresentar uma intuição sobre a diferença entre algoritmos de tempo linear, como é o caso do *Counting Sort*, e algoritmos de tempo polinomial, como é o caso do *Bubble Sort*. Veja que as descrições são superficiais e repletas de imprecisões, pois os detalhes desse tipo de análise não fazem parte do escopo desta disciplina.

O algoritmo *Counting Sort* ordena um arranjo de entrada em tempo linear, porém não se pode ignorar as premissas que possibilitam esse desempenho. O arranjo de entrada deve ser composto por valores inteiros na faixa de 0 até k . Essa premissa pode inviabilizar a utilização do algoritmo em determinados contextos.

Uma aplicação particular do algoritmo *Counting Sort* é quando temos que ordenar números compostos por vários dígitos. Essa estratégia era utilizada para a ordenação de cartões perfurados nos primórdios da computação. A estratégia consiste em ordenar cada dígito do número individualmente, iniciando pelo dígito menos representativo.

A ordenação digital considera algum algoritmo de ordenação estável para ordenar cada dígito. Desse modo, quando a ordenação for feita sobre o próximo dígito mais representativo e esses forem iguais, a ordenação sobre o dígito anterior garante que os elementos ocupem as posições corretas.

A Figura 3 apresenta um exemplo de ordenação digital, no qual há a ordenação dos números que compõem o conjunto a partir do dígito menos representativo. No primeiro de três passos há a ordenação pelo dígito mais à direita, se pode considerar qualquer algoritmo de ordenação estável para essa tarefa, porém veja que o fato de os dígitos estarem compreendidos entre 0 e 9 resulta nas premissas necessárias para a aplicação do algoritmo *Counting Sort*.

351		35 1		5 1 8		0 76
127		32 2		3 2 2		1 27
322		73 3		1 2 7		2 28
518	➔	07 6	➔	2 2 8	➔	3 22
228		12 7		7 3 3		3 51
733		51 8		3 5 1		3 88
076		22 8		0 7 6		5 18
388		38 8		3 8 8		7 33

Figura 3 –Etapas da ordenação digital

Fonte: Adaptado de CORMEN, 2009



Importante!

Um algoritmo de ordenação é estável quando a ordem original de elementos de mesmo valor é mantida após a execução do algoritmo. O algoritmo *Counting Sort* é estável.

Apesar de, aparentemente, o algoritmo *Counting Sort* ser adequado para a ordenação digital, seu processo de execução prevê a ordenação em outro arranjo, isso pode ser uma característica desfavorável à sua aplicação. Há de se considerar diferentes aspectos associados ao contexto da implementação para a escolha do algoritmo de ordenação ideal.

Ordenação por Balde (*Bucket Sort*)

Assim como ocorre no algoritmo de ordenação por contagem, ou *Counting Sort*, o algoritmo de ordenação por balde, ou *Bucket Sort*, também considera ou assume algo sobre a entrada. Nesse caso, o *Bucket Sort* considera que a entrada é um arranjo no intervalo $[0,1)$ distribuído uniformemente, ou seja, os elementos no arranjo ocorrem com a mesma probabilidade. A notação anterior para o intervalo indica que o valor 0 pertence ao intervalo e o valor 1 não pertence. Veja que o arranjo A , descrito a seguir, apresenta essas características.

$$A = [0.78; 0.12; 0.45; 0.47; 0.18; 0.63; 0.72; 0.42; 0.65; 0.27]$$

O algoritmo *Bucket Sort* cria $|A|$ baldes ou buckets no intervalo $[0,1)$, onde $|A|$ é o número de elementos existentes no arranjo A , assim, para o arranjo A teríamos 10 buckets. O algoritmo 2 descreve as instruções do algoritmo *Bucket Sort*. A primeira instrução do algoritmo descreve a criação de um arranjo B com 10 posições, representando os 10 buckets (linha 1).

Cada posição no arranjo B é uma lista, inicialmente vazia, que vai receber os elementos do arranjo A (linhas 2 e 3). Veja a unidade IV desta disciplina a estrutura de dados lista ligada, que é utilizada pelo algoritmo *Bucket Sort*. Os valores no arranjo A são utilizados para a definição da lista que os receberão. Considerando que os valores em A estão no intervalo $[0,1)$ e que o número de buckets é igual ao número de elementos em A , o algoritmo considera $\lfloor |A| \times A[i] \rfloor$ como indicação da lista B . Esta notação indica que a lista utilizada para a inserção do elemento em $A[i]$ é indicada pelo produto do tamanho do arranjo de entrada pelo valor arredondado para baixo. Assim cada valor em A será inserido em uma lista B específica, como descrito a seguir.

$$\begin{aligned} A[1] = 0.78 &\Rightarrow 0.78 \times 10 = 7.8 \Rightarrow \lfloor 7.8 \rfloor = 7 \Rightarrow B[7] \leftarrow A[1] \\ A[2] = 0.12 &\Rightarrow 0.12 \times 10 = 1.2 \Rightarrow \lfloor 1.2 \rfloor = 1 \Rightarrow B[1] \leftarrow A[2] \\ A[3] = 0.45 &\Rightarrow 0.45 \times 10 = 4.5 \Rightarrow \lfloor 4.5 \rfloor = 4 \Rightarrow B[4] \leftarrow A[3] \\ A[4] = 0.47 &\Rightarrow 0.47 \times 10 = 4.7 \Rightarrow \lfloor 4.7 \rfloor = 4 \Rightarrow B[4] \leftarrow A[4] \\ A[5] = 0.18 &\Rightarrow 0.18 \times 10 = 1.8 \Rightarrow \lfloor 1.8 \rfloor = 1 \Rightarrow B[1] \leftarrow A[5] \\ A[6] = 0.63 &\Rightarrow 0.63 \times 10 = 6.3 \Rightarrow \lfloor 6.3 \rfloor = 6 \Rightarrow B[6] \leftarrow A[6] \\ A[7] = 0.72 &\Rightarrow 0.72 \times 10 = 7.2 \Rightarrow \lfloor 7.2 \rfloor = 7 \Rightarrow B[7] \leftarrow A[7] \\ A[8] = 0.42 &\Rightarrow 0.42 \times 10 = 4.2 \Rightarrow \lfloor 4.2 \rfloor = 4 \Rightarrow B[4] \leftarrow A[8] \\ A[9] = 0.65 &\Rightarrow 0.65 \times 10 = 6.5 \Rightarrow \lfloor 6.5 \rfloor = 6 \Rightarrow B[6] \leftarrow A[9] \\ A[10] = 0.27 &\Rightarrow 0.27 \times 10 = 2.7 \Rightarrow \lfloor 2.7 \rfloor = 2 \Rightarrow B[2] \leftarrow A[10] \end{aligned}$$

As inserções, descritas anteriormente, são resultados das instruções nas linhas 4 e 5 no Algoritmo 2. Note que existem valores em A que são mapeados para as mesmas listas, como os valores $A[3]$, $A[4]$ e $A[8]$ que são inseridos na lista $B[4]$, ou ainda, os valores $A[2]$ e $A[5]$, mapeados para a lista $B[1]$. O algoritmo *Bucket Sort* soluciona essas colisões de diferentes valores mapeados para uma mesma lista, ordenando-os após a finalização das inserções, como descrito nas linhas 6 e 7 do algoritmo 2. Porém muitos valores mapeados para poucas listas diminuem a eficiência do algoritmo, tornando-o tão eficiente quanto o *Insertion Sort* que é utilizado para a ordenação de cada lista. Nesse sentido, a distribuição dos valores no arranjo de entrada A é determinante da eficiência do *Bucket Sort*.

Algoritmo 2: Algoritmo *Bucket Sort*

BucketSort(A)

1. **faça** $B[0 \dots |A|-1]$ um novo arranjo
2. **para** $i \leftarrow 0$ **até** $|A|-1$ **faça**
3. **faça** $B[i]$ uma lista vazia
4. **para** $i \leftarrow 1$ **até** $|A|$ **faça**
5. insira $A[i]$ na lista $B[\lfloor |A| \times A[i] \rfloor]$
6. **para** $i \leftarrow 0$ **até** $|A|-1$ **faça**
7. ordene $B[i]$ com ordenação por inserção
8. concatene as listas $B[0]$, $B[1]$, ..., $B[|A|-1]$ em ordem

Fonte: Adaptado de ormen, 2009

Finalmente, após os valores em A terem sido distribuídos nas listas $B[i]$, o algoritmo *Bucket Sort* concatena estas listas em ordem, conforme descrito nas linhas 7 e 8 do algoritmo 2.

A Figura 4 ilustra a ordenação do arranjo A pelo algoritmo *Bucket Sort*. Veja que dos dez *buckets* somente cinco receberam os valores de A . Isso ocorre devido à distribuição dos valores em A (quanto mais uniformemente distribuídos os valores são, mais *buckets* serão ocupados com menos valores em cada *bucket*). Muitos valores em um único *bucket* demanda mais tempo para sua ordenação a partir do algoritmo *Insertion Sort*.

Os algoritmos de ordenação em tempo linear ao tamanho da entrada, como o *Counting Sort* e o *Bucket Sort*, são sensíveis às premissas sobre a entrada e não oferecem o mesmo desempenho quando as premissas não são observadas.

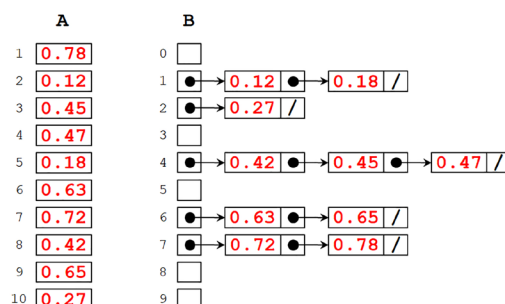


Figura 4 – Ordenação do arranjo A por meio do algoritmo *Bucket Sort*

Fonte: Adaptado de CORMEN, 2009

No caso do *Bucket Sort* a principal premissa é a distribuição uniforme dos elementos no arranjo de entrada. Veja que a pertinência ao intervalo $[0,1]$ pode ser obtida a partir da normalização dos elementos no arranjo.

O estudo da eficiência dos algoritmos é uma área da ciência da computação que tem por objetivo classificar os algoritmos em função do tempo que é consumido para a obtenção do resultado. Na seção seguinte, vamos explorar, de maneira preliminar, a análise de complexidade de algoritmos de modo a obter uma intuição inicial sobre o desempenho de diferentes algoritmos.

Fundamentos de Análise de Complexidade de Algoritmos

A base da análise dos algoritmos está na contagem do número de instruções que são executadas por eles. Cada instrução demora certo tempo para ser realizada, assim, quanto maior for o número de instruções, tanto maior será o tempo de execução. Para generalizar nossa análise, vamos considerar que as instruções demorem um mesmo tempo para serem executadas, apesar de sabermos que isso de fato não é real e que as características de hardware impactam no tempo de execução das instruções.

Considere o exemplo a seguir, descrito em pseudocódigo:

1. **para** $i \leftarrow 1$ **até** n **faça**
2. $V[i] \leftarrow i$.

A linha 1 contém um laço que será executado $n+1$ vezes. Esse tipo de laço inicia com uma atribuição de valor à variável de controle, no caso $i \leftarrow 1$, e uma verificação da condição de parada do laço que é implícita na representação considerada mas podemos pensar em uma condição como $i \leq n$ para esse exemplo. Após a inicialização da variável e da verificação da condição de parada, os próximos passos serão o incremento da variável de controle e a verificação da condição de parada do laço.

Supondo que $n = 3$, os valores assumidos pela variável $i = 1, 2$ e 3 avaliarão como verdadeira a expressão condicional de controle, permitindo que as instruções pertinentes ao corpo do laço sejam executadas, também, 3 vezes. Note que a variável i ainda será incrementada uma última vez, assumindo o valor 4 e avaliando a expressão condicional $i \leq n$ como falsa, interrompendo a execução das instruções no corpo do laço. Assim, a instrução na linha 1 será executada $n + 1$ vezes e a instrução na linha 2 será executada n vezes, totalizando $n + 1 + n = 2n + 1$ instruções executadas.

Lembre-se que estamos considerando que todas as instruções demoram o mesmo tempo para serem executadas. Nesse contexto, se supormos que a execução de uma instrução demore 0,001 segundo e que $n = 1000$ a execução das linhas 1 e 2 do exemplo, levará 2,001 segundos. Considere outra instância cujo $n = 1000000$, para essa instância o tempo de execução passaria para 2000,001 segundos, ou aproximadamente 33,33 minutos.

Nos exemplos anteriores podemos observar que o tempo de execução das linhas 1 e 2 cresce em função do aumento de n , ou seja, a medida que o tamanho da instância aumenta o tempo de execução, também, aumenta. Assim, podemos observar que a análise do tempo de execução de algoritmos pode ser feita em função do tamanho da instância de entrada. Essa análise não é completa pois parece natural que instâncias grandes demorem mais para serem executadas que instâncias pequenas e, assim, não se tem de fato uma conclusão para que se possa classificar o algoritmo como eficiente ou não.

O laço anterior considerado como exemplo executa $2n + 1$ instruções, essa observação está alinhada com a descrição anterior na qual afirmamos que a análise do tempo de execução de algoritmos é feita em função do tamanho da instância. Veja que a expressão que descreve o número de instruções executadas é apresentada em função de n que representa o tamanho da instância de entrada. Assim, podemos dizer que o tempo de execução do laço é definido por $T(n) = 2n + 1$, ou seja, o tempo de execução do laço é proporcional ao número de instruções executadas.

Para uma instância $n = 1000$ o número de instruções executadas é $T(n) = 2001$. Supondo que a instância dobre de tamanho, ou seja $n = 2000$, teremos $T(n) = 4001$ instruções. Novamente, dobrando o tamanho da instância $n = 4000$, o número de instruções é $T(n) = 8001$. Note que sempre que dobramos o tamanho da instância, o número de instruções também dobra, ou seja, o número de instruções aumenta de forma linear ao tamanho da instância.



Na equação $T(n) = 2n+1$ o que determina o crescimento do número de instruções executadas?

O crescimento do número de instruções é dependente da variável n , sempre que n cresce $T(n)$ também cresce de forma linear. Assim, podemos dizer que o crescimento do número de instruções executadas no laço é definido por $T(n) = n$, ou seja, o coeficiente que multiplica n e a unidade adicional, na expressão original $T(n) = 2n+1$, não definem o comportamento da função para valores grandes de n .

Os algoritmos *Counting Sort* e *Bucket Sort* são algoritmos de ordenação em tempo linear, ou seja, o número de instruções executadas é proporcional a $T(n) = n$. Isso significa que o tempo de execução aumenta linearmente com o número de elementos a serem ordenados. Algoritmos lineares são muito bons, do ponto de vista da eficiência computacional. Para ilustrar a eficiência dos algoritmos lineares, vamos considerar outro exemplo:

1. para $i \leftarrow 1$ até n faça
2. para $j \leftarrow 1$ até n faça
3. $V[i][j] \leftarrow i+j$

Seguindo o mesmo procedimento adotado para o exemplo anterior, iniciaremos com a contagem de instruções executadas em cada linha do algoritmo. A linha 1 será executada $n+1$ vezes, da mesma forma analisada para o primeiro exemplo. O número de vezes que a linha 2 será executada é dado por $\sum_{i=1}^n (n+1)$, esse somatório indica que para $i = 1, 2, \dots, n$ o laço na linha 2 é executado $n+1$ vezes, ou seja $n(n+1)$ vezes.

Por fim, a linha 3 será executada $\sum_{i=1}^n n$ vezes, ou seja $n \times n$ vezes. Somando os totais de cada linha do algoritmo, temos:

$$T(n) = (n+1) + n(n+1) + n \times n$$

$$T(n) = n+1+n^2+n+n^2$$

$$T(n) = 2n^2+2n+1$$

Aplicando a mesma lógica, considerada para o primeiro exemplo, podemos verificar qual é o comportamento da equação cada vez que o tamanho da instância dobra.

Tabela 1 – Comparação entre o crescimento do número de instruções e o tamanho da instância de entrada para o segundo exemplo

n	$2n^2+2n+1$	$2n^2$	n^2
1.000	2.002.001	2.000.000	1.000.000
2.000	8.004.001	8.000.000	4.000.000
4.000	32.008.001	32.000.000	16.000.000
8.000	128.016.001	128.000.000	64.000.000

A Tabela 1 apresenta o número de instruções que são executadas para diferentes tamanhos de instâncias, considerando, também, diferentes expressões. Ao compararmos as duas primeiras colunas, é possível observar o número exato de instruções executadas para uma instância inicial de tamanho $n = 1.000$ e o aumento no número de instruções à medida que dobramos o tamanho da entrada.

A primeira observação importante é que o crescimento do número de instruções não é linear ao tamanho da entrada. Veja que para n igual a um mil, o número de instruções é superior a dois milhões. Se dobrarmos o tamanho da entrada, o número de instruções é multiplicado por quatro, chegando a mais de oito milhões de instruções.

Note que a terceira coluna representa o número de instruções a partir da variável com maior expoente, ou seja $2n^2$. Nesse caso, os valores absolutos não sofreram grande impacto, permitindo a conclusão de que os membros $2n+1$, de fato, não são determinantes do crescimento do número de instruções executadas.

Estamos, de fato, interessados no padrão de crescimento do número de instruções executadas a partir do tamanho da instância. Note que as colunas identificadas com $2n^2$ e n^2 , apesar de apresentarem valores absolutos distintos, descrevem o mesmo padrão de crescimento para o número de instruções. Em ambos os casos, sempre que a entrada dobra de tamanho o número de instruções executadas quadruplica. Assim, podemos assumir que o tempo de execução para as instruções do segundo exemplo é proporcional a $T(n) = n^2$.

Os algoritmos cujo tempo de execução é $T(n) = n^2$ têm o número de instruções executadas dado por uma função quadrática de n . Os algoritmos de tempo quadrático são menos eficientes que os algoritmos de tempo linear. Os algoritmos *Bubble Sort* e *Insertion Sort* são algoritmos de tempo quadrático, portanto, são menos eficientes que os algoritmos *Counting Sort* e *Insertion Sort*.

O tempo que um algoritmo demora em terminar sua execução está ligado ao tamanho de sua entrada. A ligação entre o tamanho da entrada e o tempo para execução é definida por uma função que descreve a forma como o tempo de execução cresce de acordo com o aumento do tamanho da entrada.

Quando analisamos o tempo de execução de um algoritmo, estamos tratando de sua complexidade, assim, a análise de complexidade de algoritmos é a área que estuda o tempo que os algoritmos demoram em finalizar sua execução. Nesse contexto, a análise de complexidade não está interessada em apresentar o número de instruções executadas ou o tempo que isso vai demorar. O objetivo da análise de complexidade é fornecer uma função que descreva o padrão de crescimento do tempo de execução dado o tamanho da instância de entrada.

Existem diferentes funções que descrevem os tempos de execução mais comuns observados para algoritmos. Veja na Tabela 2 algumas funções e os respectivos números de instruções executadas a partir do tamanho da entrada. Veja que os números de instruções apresentados na tabela é somente para que se possa ter uma intuição do padrão de crescimento do tempo em função da entrada.

Tabela 2 – exemplos de funções que descrevem o padrão de crescimento do tempo de execução dos algoritmos em função do tamanho da instância de entrada

n	$T(n) = n$	$T(n) = n^2$	$T(n) = n^3$	$T(n) = 2^n$	$T(n) = n!$
2	2	4	8	4	2
4	4	16	64	16	24
8	8	64	512	256	40.320
10	10	100	1.000	1.024	3.628.800

A contagem de instruções executadas em cada linha do algoritmo é uma tarefa que permite calcular de maneira precisa o número de instruções que são executadas por um algoritmo. Na maior parte das vezes, estamos interessados em tamanhos de entrada grandes e, para esses casos, não importa a precisão do número de instruções, mas, sim, o padrão de crescimento desse número. Por conta disso, as constantes multiplicativas e os termos de ordem mais baixa são desconsiderados quando tratamos com entradas grandes cujo número de instruções executadas será dominado pelo termo de ordem mais alta, como foi o caso no exemplo cujo número de instruções é dado por $2n^2 + 2n + 1$. Veja que nesse exemplo desconsideramos os termos de ordem mais baixa n^1 e n^0 e a constante multiplicadora 2, restando apenas n^2 , que descreve o padrão de crescimento do tempo de execução.

Denominamos eficiência assintótica a análise do tempo de execução dos algoritmos, quando o tamanho da instância de entrada é suficientemente grande, de modo que apenas a ordem de crescimento do tempo de execução seja relevante.

Nessa seção, tratamos da eficiência dos algoritmos, sob a ótica do tempo de execução, de maneira superficial e omitindo detalhes relevantes nesse tipo de análise. O intuito aqui é fornecer uma intuição sobre o assunto, o qual será objeto de estudo em outra disciplina do curso.



Em Síntese

- Os algoritmos *Counting Sort* e *Bucket Sort* são algoritmos de ordenação em tempo linear;
- Um algoritmo de ordenação em tempo linear indica que o tempo de execução cresce, de maneira linear, com o tamanho da instância de entrada;
- Os algoritmos em tempo linear são considerados muito eficientes;
- O algoritmo *Counting Sort* considera que a entrada é composta por inteiros menores ou iguais a k ;
- O algoritmo *Bucket Sort* considera que a entrada é composta por valores no intervalo $[0,1)$ distribuídos de maneira uniforme;
- A análise da eficiência assintótica dos algoritmos objetiva identificar a ordem de crescimento dos respectivos tempos de execução para entradas suficientemente grandes.

Material Complementar

Indicações para saber mais sobre os assuntos abordados nesta Unidade:

🎥 Vídeos

Sorting Algorithm | Counting Sort – Step by step guide

<https://youtu.be/TTnvXY82dtM>

Sorting Algorithm | Bucket Sort – step by step guide

<https://youtu.be/geVylsFpxUs>

Sorts 2018 – Horizontal Pyramid

<https://youtu.be/vmT3XUBoxiQ>

Complexidade de algoritmos I Noção intuitiva

<https://youtu.be/KVIGx-9Cu04>

Referências

CORMEN, T. H. *et al.* **Introduction to algorithms**. MIT press, 2009.

TOSCANI, L. V. **Complexidade de algoritmos**. V. 13: UFRGS. 3. Porto Alegre Bookman 2012. (*e-book*)

ZIVIANI, N. **Projeto de algoritmos**: com implementações em JAVA e C. São Paulo Cengage Learning 2012. (*e-book*)



Cruzeiro do Sul
Educatonal